



SSW 567

Software Testing, Quality Assurance and Maintenance

Slides are based on content originally prepared by Linda Laird, and include updates by Jim Rowland, Rich Kempinski, Raz Sarimi, and a significant contribution from

Dr. Y. Morgan

02- Model-Based Testing Methods DT, *k*-Maps, and Grammars.

*Too many test-cases to execute,
too little time to do it.*

Modeling

Create a Legos replica before building the actual bridge



What is a Model?

"A *model* is a representation that is simpler than the artifact it represents but preserves (or at least approximates) some important attributes of the actual artifact."

How does it help in testing?

The model of the SUT (System Under Test) can be used as the testing Oracle, which may be complete or incomplete. Typically, a single model would be incomplete.

A model-based testing can guide us into what to test, and how to create test-cases.

What is Model-Based Testing (MBT)?



* Automatic Model-based test generation from UML state machines
from Model-based Testing for Embedded Systems, 2012, CRC Press

- Testing the SUT may follow known formal models created from the SUT's requirements or implementation.
- Use a formal model of the SUT along with test generation tools to automatically generate relevant test-cases.
- Test-case results from the SUT are compared against the Model's expected results. In other words, the **Model becomes the Oracle**.
- Models may be *formal* or *informal*.

Strategies for Building Models

- Models form a mental representation of the system's functionality:
 - Understand the system's behavior before applying modeling.
 - Must understand inputs, outputs, and actions to build a model for testing purposes.
- The description needs to be written down in an easy-to-understand form and should be as formal as practical:
 - Useful models can make test generation easier and automatable.
 - Many formal models are required to describe the system behaviors.
- Large, complex systems may require the team to use formal modeling techniques.



Somethings are actually black or white models,

,

,

or grey ;)



Types of Approaches

A) Functional, or Behavioral Approach

- Models consider the **behavior** of the system, i.e. what the system does; not how it does it.
- Needed for testing

BLACK

B) Structural Approach

- Models consider the **structure** of the system, i.e. what the system is?
- Typically works as a proof of concept.
- Describes the components, functionality, and interactions.

White



Functional Modeling

- Modeling how the system is used regardless of how it is built.
(Modeling the functionality of the system *that will be tested.*)

Components:

Behavior modeling - captures the "sequence of actions and responses" taken in doing something useful on the system.

Data modeling – captures the "*concrete* test data values" needed to execute test-cases on the system.



Use of Model-Based Methods

■ Advantages?

- Helps identify all possible and valid states/values/combinations.
- Checking for consistency and completeness.
- Help to identify test-cases.
- Help to identify requirements and design flaws.
- Potentially generate test-cases automatically.

■ Disadvantages?

- Require formality and thought (is this a plus or a minus?)
- Upfront time required to create the test model.
- Scaling can be massive (e.g. 2^n for decision trees).
It's up to the test engineer to intelligently reduce the model space.



Grammars



Grammars

- Grammars are a way of formally specifying the structure of languages.
- Grammars are used extensively in Computer Science for formal languages and protocol processing
 - E.g., programming languages, compilers, interpreters, scripts, ...
- If you don't know what grammars are,
 - You may want to do some introductory computer science reading on compilers and grammars. They are interesting and powerful.
- Backus-Naur Form (BNF) is a concise description of a language grammar.
 - Gentle introduction to BNF: <<https://www.youtube.com/watch?v=B4wMpOhOako>>
- Can also use grammars to model systems behaviors.



Using Grammars

- Grammars - used to describe inputs to a system
- However, Grammars can also be used to generate test data

Use cases:

- Language-based apps (compilers, interpreters, ...)
- Protocol-based implementations

Example: Triangle program

- Input ::= Side1 Side2 Side3
- each Side ::= Positive_number
- Number ::= [0,1,2,3,4,5,6,7,8,9]+

**Are Grammars
Black or White?**



Common Black-Box and Model-Based Testing

- **Equivalence Classes**
 - One value represent the class
- **Boundary Value Analysis**
 - Informal methods
 - Depend on skills, expertise
- **Cause-effect Graphing**
 - Visual, but can get quit complex
- **Decision tables**
 - Reduce number of test cases by "do not care" conditions
- **Pair-wise testing**
 - Test-case generation can be automated: all-pairs
- **Orthogonal Arrays**
 - Mathematically independent
- **State Transition Diagrams**
 - Formal Methods: enables checking for completeness and inconsistencies of spec's
- **Domain testing**
 - Combine and formalize these
- **Error Guessing or Special Values**
 - Special values testing



Decision Tables

Analyzing Combinations of Data Values

*Too many test-cases to execute,
too little time to do it.*

Decision Tables (DT)

- Decision Tables (DT) provide a useful basis for designing test-cases.
- DT links CONDITIONS (or causes) and ACTIONS (or effects) in a matrix.
 - Think along the lines of "if this, then that".
 - CONDITION *leads-to* → ACTION.
- DTs model complex decision logic:
(tracks complex combinations of conditions and resulting actions)
- It is a formal, table-based notation which can be processed automatically to check for qualities like consistency and completeness.
- Decision Tables obtain their power from logical expressions
(**NOT**, **AND**, **OR**, **XOR**)



Decision Tables

A Decision Table consists of:

- **Condition Stub:**
A list of named conditions predicates).
- **Condition Entry:**
 - **T** if value of condition is true
 - **F** if value of condition is false
 - **X** or **--** if value of condition is immaterial (don't care).
- **Action Stub:** A list of named actions that will be taken if a rule is satisfied.
- **Action Entry:**
 - **T** if the action takes place.
 - **F** if the action does not take place.

Stub	Entries							
	1	2	3	4	5	6	7	8
Condition								
Cond. 1	T	T	T	T	F	F	F	F
Cond. 2	T	T	F	F	T	T	F	F
Cond. 3	T	F	T	F	X	F	--	F
Actions								
Action 1	√			√				√
Action 2		√				√		√

General Mechanism

- Divide specification into "workable" pieces,
- Define all CONDITIONS and ACTIONS from the specification.
 - A CONDITION is an input / equivalence class of input conditions.
 - An ACTION is an output or a system state of the program.
 - DT focuses on situations of "if this CONDITION is true, that ACTION is taken".
- The semantic contents of the specification is analyzed and transformed into a Boolean table linking the CONDITIONS and ACTIONS.
- The columns in the final Decision Table are converted into test-cases.

	Stub	Entries							
Condition		1	2	3	4	5	6	7	8
	Condition 1	True				False			
	Condition 2	True		False		True		False	
	Condition 3	T	F			T	F		
		...	...	...	...	...	...	...	...
Action									
	Action 1	√	√		√			√	
	Action 2	√			√	√			
	Action 3		√			√	√		√
	Action 4			√				√	√
		...	...	...	...	...	...	...	...



Example 1: Payroll Taxes

	Rule 1	Rule 2	Rule 3	Rule 4
Conditions				
C1: Resident?	Y	Y	N	N
C2: Gross > 15 K	Y	N	Y	N
Actions				
A1: 4% Tax?	N	✓	✓	✓
A2: 6% Tax?	✓	N	N	N

<u>CONDITION</u>	<u>ACTION</u>
C1: Not Resident? C2: Resident and Gross > \$15,000	A1: 4% tax A2: 6% tax



Payroll Tax Decision Tables

- Reading of Rule 2
 - IF all of the following conditions are TRUE
 - the employee is a resident indeed; and,
 - his gross income is NOT more than 15 K.
 - THEN the following action takes place:
 - Charge 4% tax.
 - AND the following action does NOT take place
 - Do not charge 6% tax.
- Every rule can represent a test-case.
- Every rule is crispy clear, unique, and unambiguous.

	Rule 1	Rule 2	Rule 3	Rule 4
Conditions				
C1: Resident?	Y	Y	N	N
C2: Gross > 15 K	Y	N	Y	N
Actions				
4% Tax?	N	✓	✓	✓
6% Tax?	✓	N	N	N



Reducing Payroll Tax Decision Table

- Reduce the Decision Table as follows:
 - Merge rules whose resulting actions are identical
 - If the corresponding logical expression can be simplified using Boolean algebra
 - In other words, use don't care rules

	Rule 1	Rule 2	Rule 3	Rule 4
Conditions				
C1: Resident?	Y	Y	N	N
C2: Gross > 15 K	Y	N	Y	N
Actions				
A1: Tax %	6%	4%	4%	4%

What is a "Do not care" Condition?

	TC1	TC2	TC3
Conditions			
Resident?	Y	Y	N
Gross > 15 K	Y	N	X
Actions			
Tax%	6%	4%	4%

<u>CONDITION</u>	<u>ACTION</u>
C1: Not Resident? C2: Resident and Gross > \$15,000	A1: 4% tax A2: 6% tax

How many rules in this table?

4

How many test-cases in this table?

3



Example 2: Non-Binary Conditions

A Decision Table with non-binary conditions

Conditions	TC1	TC2	TC3	TC4
X: {0-1, 2-10, 11-100, 101-1000} <i>Think of it as: (X₁, X₂, X₃, X₄)</i>	X ₁	X ₂	X ₃	X ₄
Y: {<5, 5, 6-7, >7} <i>Think of it as: (Y₁, Y₂, Y₃, Y₄)</i>	Y ₁	Y ₂	Y ₃	Y ₄
Actions				
A1: DO	L	M	N	~L

Q: Is this a condition or an Eq. Class?

How many rules?

4 x 4 = 16

Q: Which black-box techniques should be employed besides the Decision Table in this example? HOW?

Steps to Create a Decision Table

1. List all conditions in the Decision Table are in the stub's CONDITION section.
2. List all anticipated actions in the table are in the stub's ACTION section.
3. Fill columns with all possible combinations in the stub's entry section. Ensure coverage.
4. Reduce columns combinations as you can.
5. Check covered combinations.
6. Determine Default Rules.

And then,

- Understand the reasons for (don't care **X**) cases
- Generate test-cases from the final reduced rules.
- If no default rules were given in specs, then all cases not addressed by explicit rules are default rules.

	Stub	Entries							
Condition		1	2	3	4	5	6	7	8
	Condition 1	True				False			
	Condition 2	True		False		True		False	
	Condition 3	T	F			T	F		
Action									
	Action 1	√	√		√			√	
	Action 2	√			√	√			
	Action 3		√			√	√		√
		...	...	...	...	...	...	...	...



Step 1: List All Conditions

- Write down the values that cause/condition can assume;
 - Cluster related conditions closer to each other.
 - Put the most dominating condition first.
 - Put multi valued conditions last.
- Check your work

		Combinations							
Conditions	Values	1	2	3	4	5	6	7	8
Cond 1	Y/N								
Cond 2	Y/N								
Cond 3	H/M/L								
Action									
Action 1									
Action 2									
Action 3									

Step 2: Add all Anticipated Actions

- Write down the values for all anticipated actions;
 - Make sure all actions are covered.
 - Note that some actions are subtle, like changing a value in a database.
- Check your work

		Combinations							
Conditions	Values	1	2	3	4	5	6	7	8
Cond 1	Y/N								
Cond 2	Y/N								
Cond 3	H/M/L								
Action									
Action 1									
Action 2									
Action 3									

Step 3: Fill Columns (Rules)

		Combinations							
Cond	Values	1	2	3	4	5	6	7	8
Cond. 1	Y/N	Y	Y	Y	Y	N	N	N	N
Cond. 2	Y/N	Y	Y	N	N	Y	Y	N	N
Cond. 3	Y/N	Y	N	Y	N	Y	N	Y	N
Actions									
Action 1		Y		Y	Y				Y
Action 2			Y				Y		Y
Action3									



		Combinations							
Cond	Values	1	2	3	4	5	6	7	8
C1	Y/N	Y	Y	Y	Y	N	N	N	N
C2	Y/N	Y	Y	N	N	Y	Y	N	N
C3	Y/N	Y	N	Y	N	Y	N	Y	N
Actions									
A1		Y		Y	Y				Y
A2		Y	Y				Y		Y
A3						N		N	

		Combinations						
Cond	Values	1	2	3	4	5	6	7
C1	Y/N	Y	Y	Y	N	N	N	N
C2	Y/N	Y	Y	N	Y	Y	N	N
C3	Y/N	Y	N	X	Y	N	Y	N
Action								
A1		Y		Y				Y
A2		Y	Y			Y		Y
A3					N		N	

Step 4: Reduce Combinations

- Find indifferent combinations and - place a 'X'.
- Join columns where columns are identical except for one condition.
- Make additional reduction on the table as you see, right now.
- But make sure the actions are the same.**

Step 5: Check Covered Combinations

- Calculate the checksum of the first table before consolidation.
- For each column of the final table, calculate the number of combinations it represents.
- An '**X**' represents as many combinations as the condition has.
- Multiply for each '**X**' down the column.
- Compare checksum of the first table with that of the final one.

**Perform on the previous table,
and then check your answer**

And here is another example



Check Specifications Consistency

- A specification is **consistent** *iff* there are no sub-rules whose conditions are the same, but their actions are different.

Rules				
Conditions	r1	r2	r3	r4
c1	T	T	T	T
c2	T	F	F	F
c3	T	T	T	F
c4	F	F	F	F
Actions				
a1	T	T	F	F
a2	F	F	T	T
Checksum	inconsistency			

Check Specifications Completeness (IN A BINARY DT)

A specification is **complete** *iff* N binary decisions expand into exactly 2^N unique sub-rules

$2^4=16$, this table has only 6 rules

		TC1	TC2	
C1	T/F	T	T	
C2	T/F	X	F	
C3	T/F	T	X	
C4	T/F	F	X	
A1		T	F	
A2		F	T	
Checksum		2	4	6

- Can you identify some of the missing rules?

Calculate Combinations (IN A NON-BINARY DT)

If all conditions are simply Y/N values: **Checksum** = $2^{\text{\# of cond.}}$

In other words, number of values multiplied going down the condition stub.

But if we have:

1 condition may take 5 values and
4 conditions may take 2 values each:

$$\text{Checksum} = 5^1 * 2^4 = 80$$

In the table below, use the values column and multiply each value going down the column, eg. $5*2*2=20$

eg. 3-2-2-20

		Comb.				
Cond.	Values	TC1	TC2	TC3	TC4	
C1	0,1,2, 3, 4	2	3	3	4	
C2	T/F	T	F	F	X	
C3	Y/N	X	Y	N	X	
Actions						
A1		√		√		
A2		√	√		√	
Checksum		2	1	1	4	8

What Is the Maximum Number of Rules for this DT?

		Combinations			
Cond.	values	TC1	TC2	TC3	TC4
C1	2, 3, 4	2	3	3	4
C2	7 Weekdays	SA	SU	MO	X
C3	T/F	X	Y	N	X
C4	S/M/W/D	S	M	D	W
C5	Y/N & NA	Y	A	Y	N
Actions					
A1		✓		✓	
A2		✓	✓		✓

What is the max number of rules that should be in this DT?

max # of rules = $3 \times 7 \times 2 \times 4 \times 3 = 504$

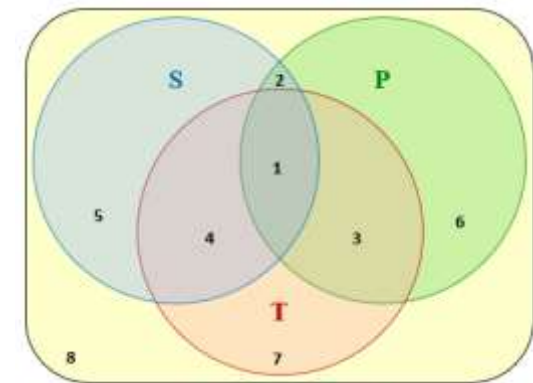
How many rules are missing in this table?

$$504 - 18 = 486$$

The combination of all missing rules is called Default Rule or Default Action

Test-case Design by Decision Tables

- Begin by examining the specification's consistency and completeness.
 - Understand all immaterial (don't care X) cases and check the expanded tables.
 - Make the default case explicit and treat it as just another set of rules for the default action.
- After the specifications have been verified for completeness and consistency, the objective of the test-cases is to:
 - show that the implementation provides the correct action for all combinations of predicate values.
- **Where do you think the Default Rule is represented on the SPT chart?**



The Decision Tables Test-case Design for White-Box

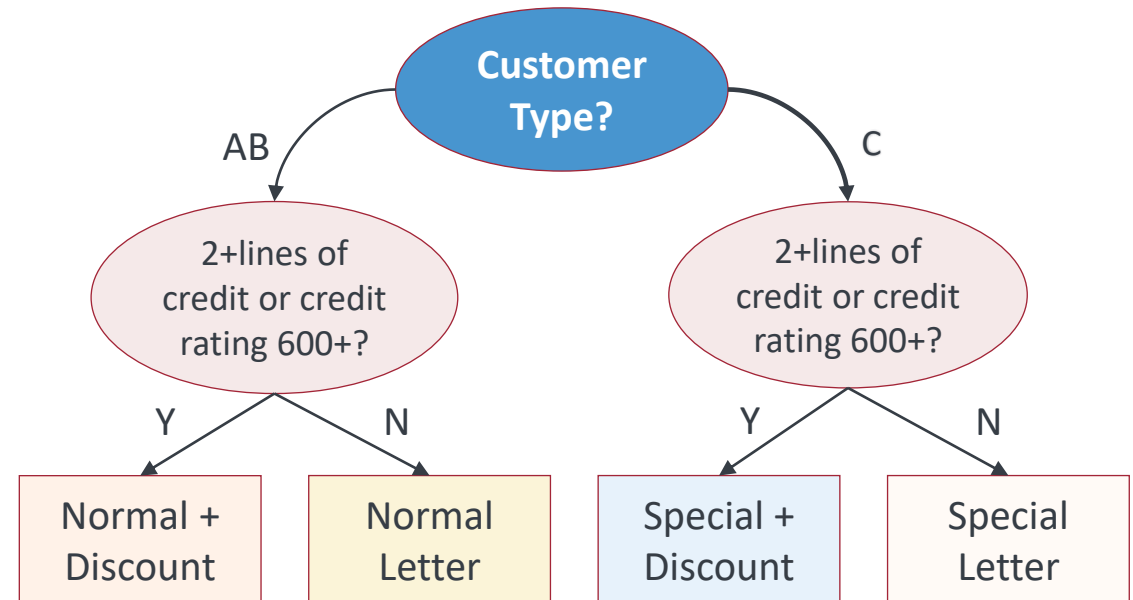
- If there are k rules all with N binary predicates, then $k = 2^N$ gives you the number of test-cases to consider.
 - Identify the places where conditions are invoked.
 - Identify the places where actions are initiated.
 - Instrument the paths from the conditions to the actions so that you can show that the correct action was invoked for each rule.



Decision Tree

Decision Tables can be represented in a tree structure

Conditions		Combinations			
C1	Customer Type	AB	AB	C	C
C2	2+Lines of Credit or Credit Rating 600+	Y	N	Y	N
Actions					
A1	Normal Letter		√		
A2	Special Letter				√
A3	Normal + Offer	√			
A4	Special + Offer			√	
Rule Count		2	1	1	14



Decision Tables Discoveries

- Ambiguity (inconsistency) = Identical rules (columns) with different actions.
Similar to inconsistency described before.
- Redundancy = identical rules (columns) and identical actions.
- Incompleteness = failure to specify all rules (columns).
- Karnaugh Maps are more concise but can get complex with complex and large systems.
- Decision Tables are recommended for logically complex systems and large systems.



Comments on Decision Tables

- A good approach for testing systems with complex conditional behavior.
- Dependencies usually yield **impossible combinations**.
- Decision Tables support extensive Consistency and Completeness Analysis.
 - If there was any **missing** or **inconsistent** condition, DT will discover it.
- Decision Tables apply to executables and compilable versions of the (sub)system.
- Decision Tables can be applied on **input** as well as **output** variables.
- Decision Tables apply to **Black-Box** as well as **White-box**.
- Its' DONOT-CARE conditions require **special attention**.





Decision Tabled Examples

Will solve as much as we can in class.



The Triangle Problem

Consider a software that reads three input data values (a, b, & c). These values represent the lengths of the three sides of a triangle. The purpose of this software is to display a message stating whether the triangle is

- scalene (i.e., no two sides are equal),
- isosceles (two sides equal), or
- equilateral (all sides equal).

As software goes, this is very low in complexity.
How do we test it?

from Glen Myers in his classic book, "The Art of Software Testing".



Example Decision Table for the Triangle Problem

One Decision Table for the Triangle Problem		
#	Condition	Using the Triangle Problem example, and using the 4 conditions, and 5 actions described here: - What are the maximum number of rules in this Decision Table? - fill-in this Decision Table
C1	a, b, c are a triangle	
C2	a=b?	
C3	b=c?	
C4	c=a?	
	Actions	
A1	not a triangle	
A2	scalene	
A3	isosceles	
A4	equilateral	
A5	Impossible	
Number of rules=		16

Did you notice that DT may suggest combinations that are impossible?

Is this good or bad?

A Second Formulation

A Second Decision Table for the Triangle Problem		
#	Condition	Using the Triangle Problem example, and using the 6 conditions, and 5 actions described here: - What are the maximum number of rules in this Decision Table? - fill-in this Decision Table
C1	$a < (b+c)$	
C2	$b < (c+a)$	
C3	$c < (a+b)$	
C4	$a=b?$	
C5	$a=c?$	
C6	$b=c?$	
Actions		
A1	not a triangle	
A2	scalene	
A3	isosceles	
A4	equilateral	
A5	impossible	
Number of rules=		

Counting Rules to Check Completeness

#	Condition	R01	R02	R03
C1	month in M_1	T	--	--
C2	month in M_2	--	T	--
C3	month in M_3	--	--	T
Rule Count				

Count the number of rules in this table

- A limited-entry Decision Table with n conditions has 2^n rules.
- A DONOT-CARE entry doubles the count of each binary rule.
- What are the possibilities when the rule count is $< 2^n$ rules? How about $> 2^n$ rules?
- More precise to use $F!$ (must be false) than -- (DONOT-CARE).



Decision Table with Mutually Exclusive Conditions

#	Condition	R01	R02	R03
C1	$M = \{M_1, M_2, M_3\}$	M_1	--	--
C2	$D = \{D_1, D_2, D_3, D_4, D_5\}$	--	D_5	--
C3	$Y = \{Y_1, Y_2\}$	--	--	Y_2
Actions				
A1				
A2				
A3				

Can you think of the relation between this table and Equivalence Class Testing?

A better way is to make one condition:

$C1 = \{M_1, M_2, M_3\}$

another solution:

$C1 = \{M_1, M_2, M_3, M_0\}$

Where: M_0 represents a situation when month is not in $\{M_1, M_2, M_3\}$.

What is the max number of rules if each of the 3 given conditions form a mutually exclusive ranges or conditions?

Redundant Decision Table

A Redundant Decision Table							
#	Condition	1-15	16	17-21	22-36	37	38
C1	$X = \{X_1, X_2, X_3, X_4\}$	X_1	X_2	X_4	X_3	X_4	X_1
C2	$Y = \{Y_1, Y_2, Y_3\}$	--	Y_1	Y_3	--	Y_2	Y_3
C3	$Z = \{Z_1, Z_2, Z_3, Z_4, Z_5\}$	--	Z_1	--	--	Z_4	Z_5
Actions							
A1		✓	✓	✓	--	--	✓
A2		--	✓	✓	✓	--	--
A3		✓	--	✓	✓	✓	✓

Are there any ambiguity, redundancy, or incompleteness in this table?

Inconsistent Decision Table

An Inconsistent Decision Table							
#	Condition	1-15	16	17-21	22-36	37	38
C1	$X = \{X_1, X_2, X_3, X_4\}$	X_1	X_2	X_4	X_3	X_4	X_1
C2	$Y = \{Y_1, Y_2, Y_3\}$	--	Y_1	Y_3	--	Y_2	Y_3
C3	$Z = \{Z_1, Z_2, Z_3, Z_4, Z_5\}$	--	Z_1	--	--	Z_4	Z_5
Actions							
A1		✓	✓	✓	--	--	--
A2		--	✓	✓	✓	--	✓
A3		✓	--	✓	✓	✓	--

Are there any ambiguity, redundancy, or incompleteness in this table?

ATM Banking System Example

- For this part the relevant causes are:
 - **C1:** The account number is correct.
 - **C2:** The signature matches.
 - **C3:** There is enough money in the account.
- The possible actions are:
 - **A1:** Give money.
 - **A2:** Give statement: "not enough money"
 - **A3:** Call the vigilance dept. & check for fraud.



ATM Banking System Decision Table

- For this part the relevant causes are:
 - **C1:** The account number is correct.
 - **C2:** The signature matches.
 - **C3:** There is enough money in the account.
- The possible actions are:
 - **A1:** Give money.
 - **A2:** Give statement: "not enough money"
 - **A3:** Call vigilance dept. & check for fraud.

What's the max number of rules?

8

#	1	2	3	4	5	6	7	8	
C1	F	F	F	F	T	T	T	T	
C2	F	F	T	T	F	F	T	T	
C3	F	T	F	T	F	T	F	T	
A1	F	F	?	?	F	?	F	T	
A2	F	F	?	?	T	?	T	F	
A3	T	T	?	?	T	?	F	F	
Σ	1	1	1	1	1	1	1	1	8

Decision Table, Reading of Rule # 2

- C3: There is enough money in the account

- THEN the following action takes place

- A3: Call the vigilance dept. & check for fraud.

- AND the following actions do NOT take place

- A1: Give money.

- A2: Give statement: "not enough money"

R2

#	1	2	3	4	5	6	7	8	
C1	F	F	F	F	T	T	T	T	
C2	F	F	T	T	F	F	T	T	
C3	F	T	F	T	F	T	F	T	
A1	F	F	?	?	F	?	F	T	
A2	F	F	?	?	T	?	T	F	
A3	T	T	?	?	T	?	F	F	
Σ	1	1	1	1	1	1	1	1	8

Can this be 'X'

Decision Tables Direct Example

Assume the following Decision Table:

- **A1:** Takes place when conditions C1, C2 are true and conditions C3, C4 are false; also, when conditions C1, C3, C4 are true.
- **A2:** Takes place when conditions C1, C2, C3, & C4 are false.
- **A3:** Takes place when condition C1 is false and condition C4 is true.

**If you only translate
the given specs,
can you draw
the DT?**

**Q: Are all possible combinations
of predicates covered?**



Ways for **X**'s to Expand

- Do not forget that immaterial cases are expanded by converting each **X** entry into a pair of entries, one with a **T** and one with a **F**.
- If a "do-not-care" value is associated with a condition that takes one of 4 values {S, M, D, W} , then you need to keep in mind that every **X** maps to 4 entries **S/M/D/W**.

Q: How many rules/TEST-CASES do we have here?

#	1	2	3	4	
C1	T	T	F	F	
C2	T	T/F	F	T/F	
C3	F	T	F	T/F	
C4	F	T	F	T	
A1	√	√	--	--	
A2	--	--	√	--	
A3	--	--	--	√	
Σ	1	2	1	4	8



More Examples



DT Example for *Mailing* Customer

Requirements:

A mailing campaign is to be sent to customers. The content of the mailing describes the current discounts and potential additional discounts. The content of the mailing depends on the types of customers:

- Customer Types A and B get a normal letter,
- Customer Type C gets a special letter,
- Any customer with two or more current lines of credit or with a credit rating of *at least 600* gets an offer with additional discounts.

Mailing DT Example: Build a Model

Are the requirements clear, concise, and complete?

Building a model may help to disambiguate.

Step 1: Start with Inputs and Outputs;

CONDITION:

C1: Customer types: {A, B, C}

C2: Current Lines of Credit ≥ 2 : {Y,N}

C3: Credit rating 600+: {Y, N}

ACTION:

A1: Letter type: {Normal, Special,

A2: Normal + Discount, Special + Discount}

#						
Customer Type	A, B, C					
2+ Lines of Credit						
Credit Rating 600+						
$\Sigma =$						

Ambiguities and Assumptions:

Does the Special Letter apply to all customer types or just customer type C?

- Assume only customer type C is eligible for the special letter,

Are there other types of customers?

- Assume No.

Are there other credit ratings or lines of credit?

- Assume No



Mailing DT Example: Total Combinations

Step 2: Calculate the total possible combinations

Inputs:

C1: Customer type: {A, B, C}

C2: Current Lines of Credit 2+: {Y,N}

C3: Credit rating 600+: {Y, N}

3 customer types

x 2 credit ratings

x 2 line of credit values

Total combinations = $3 \times 2 \times 2 = 12$

#					
Customer Type	A, B, C				
2+ Lines of Credit					
Credit Rating 600+					
$\Sigma =$					12



Step 3.1: Fill in the Inputs with All Combinations

CONDITIONS	VALUES	COMBINATIONS											
		1	2	3	4	5	6	7	8	9	10	11	12
Customer Type	A, B, C	A	A	A	A	B	B	B	B	C	C	C	C
2+ Lines of Credit	Y, N	Y	Y	N	N	Y	Y	N	N	Y	Y	N	N
Credit Rating 600+	Y, N	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N
ACTIONS													
Normal Letter	A1												
Special Letter	A2												
Nrml. Ltr + Discount	A3												
Spec. Ltr + Discount	A4												
	Σ	1	1	1	1	1	1	1	1	1	1	1	12

Step 3.2: Fill in the Actions Using Requirements

CONDITIONS	VALUES	COMBINATIONS											
		1	2	3	4	5	6	7	8	9	10	11	12
Customer Type	A, B, C	A	A	A	A	B	B	B	B	C	C	C	C
2+ Lines of Credit	Y, N	Y	Y	N	N	Y	Y	N	N	Y	Y	N	N
Credit Rating 600+	Y, N	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N
ACTIONS													
Normal Letter	A1				Y				Y				
Special Letter	A2												Y
Nrml. Ltr + Discount	A3	Y	Y	Y		Y	Y	Y					
Spec. Ltr + Discount	A4									Y	Y	Y	
	Σ	1	1	1	1	1	1	1	1	1	1	1	12

Now we have unambiguous, complete specification that can be reviewed with the stakeholder to ensure it is what was intended – and can be turned into test-cases easily.

Step 4.1: Reducing from 12 to 8

CONDITIONS	VALUES	COMBINATIONS											
		1	2	3	4	5	6	7	8	9	10	11	12
Customer Type	A, B, C	A	A	A	A	B	B	B	B	C	C	C	C
2+ Lines of Credit	Y, N	Y	Y	N	N	Y	Y	N	N	Y	Y	N	N
Credit Rating 600+	Y, N	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N
ACTIONS													
Normal Letter	A1				Y				Y				
Special Letter	A2												Y
Nrml. Ltr + Discount	A3	Y	Y	Y		Y	Y	Y					
Spec. Ltr + Discount	A4									Y	Y	Y	
	Σ	1	1	1	1	1	1	1	1	1	1	1	12

Reduce

1 & 5

2 & 6

3 & 7

4 & 8

But we have the same outcomes for A and B, regardless of the values of the other conditions, so we can combine the values for A and B and reduce the size of the matrix.

Step 4.2: Reduced from 12 Columns to 8

CONDITIONS	VALUES	COMBINATIONS							
		1	2	3	4	5	6	7	8
Customer Type	A, B, C	AB	AB	AB	AB	C	C	C	C
2+ Lines of Credit	Y, N	Y	Y	N	N	Y	Y	N	N
Credit Rating 600+	Y, N	Y	N	Y	N	Y	N	Y	N
ACTIONS									
Normal Letter	A1				Y				
Special Letter	A2								Y
Nrml. Ltr + Discount	A3	Y	Y	Y					
Spec. Ltr + Discount	A4					Y	Y	Y	
	Σ	2	2	2	2	1	1	1	1

Renumbered

12

Same Total

The model shows that we have the same actions for A & B, so, we collapse the A & B values and reduce the size of the matrix from 12 combinations to 8 combinations.

Step 4.2: Reducing from 8 to 6

	VALUES	COMBINATIONS								
CONDITIONS		1	2	3	4	5	6	7	8	
Customer Type	A, B, C	AB	AB	AB	AB	C	C	C	C	
2+ Lines of Credit	Y, N	Y	Y	N	N	Y	Y	N	N	
Credit Rating 600+	Y, N	Y	N	Y	N	Y	N	Y	N	
ACTIONS										
Normal Letter	A1				Y					
Special Letter	A2								Y	
Nrml. Ltr + Discount	A3	Y	Y	Y						
Spec. Ltr + Discount	A4					Y	Y	Y		
	Σ	2	2	2	2	1	1	1	1	12

Customer Type == AB with 2+ Lines of credit have the same action regardless of their Credit Rating.

Step 4.3: Reduced to 6 Columns

CONDITIONS	VALUES	COMBINATIONS					
		1	2	3	4	5	6
Customer Type	A, B, C	AB	AB	AB	C	C	C
2+ Lines of Credit	Y, N	Y	N	N	Y	N	N
Credit Rating 600+	Y, N	--	Y	N	--	Y	N
ACTIONS							
Normal Letter	A1			Y			
Special Letter	A2						Y
Nrml. Ltr + Discount	A3	Y	Y				
Spec. Ltr + Discount	A4				Y	Y	
	Σ	4	2	2	2	1	1

Renumbered

12

Same Total

We've reduced the original 12 columns (test-cases) to 6. Can we reduce further?

Step 4.3: Reducing from 6 to 4

1	VALUES	COMBINATIONS						
CONDITIONS		1	2	3	4	5	6	
Customer Type	A, B, C	AB	AB	AB	C	C	C	
2+ Lines of Credit	Y, N	Y	N	N	Y	N	N	
Credit Rating 600+	Y, N	--	Y	N	--	Y	N	
ACTIONS								
Normal Letter	A1			Y				
Special Letter	A2						Y	
Nrml. Ltr + Discount	A3	Y	Y					
Spec. Ltr + Discount	A4				Y	Y		
	Σ	4	2	2	2	1	1	12

Customer Type == AB and (2+ Lines of Credit == Y OR Credit Rating 600+ == Y) have same action.

Step 4.4: Reduced to 4 Columns

1

CONDITIONS	VALUES	COMBINATIONS			
		1	2	3	4
Customer Type	A, B, C	AB	AB	C	C
2+ Lines of Credit <u>OR</u> Credit Rating 600+	Y, N	Y	N	Y	N
ACTIONS					
Normal Letter	A1		Y		
Special Letter	A2				Y
Nrml. Ltr + Discount	A3	Y			
Spec. Ltr + Discount	A4			Y	
	Σ	6	2	3	1

Renumbered

Special Attention
This step require special attention. When combining rules 1&2 and 4&5 from the prev. slide, we also combined conditions C2 & C3. Hence, the number of true rules for column 1 & 3 in this slide became tricky to count.

12

Same Total

No more reduction is possible. We've reduced the model from 12 combinations to 4 and fully specified the model without ambiguity! The model is now smaller and easier to understand.

Verify Checksums (INITIAL Table, Step 4.1)

	VALUES	COMBINATIONS												
CONDITIONS		1	2	3	4	5	6	7	8	9	10	11	12	
Customer Type	A, B, C	A	A	A	A	B	B	B	B	C	C	C	C	
2+ Lines of Credit	Y, N	Y	Y	N	N	Y	Y	N	N	Y	Y	N	N	
Credit Rating 600+	Y, N	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N	
ACTIONS														
Normal Letter	A1				Y				Y					
Special Letter	A2												Y	
Nrml. Ltr + Discount	A3	Y	Y	Y		Y	Y	Y						
Spec. Ltr + Discount	A4									Y	Y	Y		
	Σ	1	1	1	1	1	1	1	1	1	1	1	1	12

Verify Checksums (FINAL Table Step 4.4)

1	VALUES	COMBINATIONS				
CONDITIONS		1	2	3	4	
Customer Type	A, B, C	AB	AB	C	C	
2+ Lines of Credit <u>OR</u> Credit Rating 600+	Y, N	Y	N	Y	N	
ACTIONS						
Normal Letter	A1		Y			
Special Letter	A2				Y	
Nrml. Ltr + Discount	A3	Y				
Spec. Ltr + Discount	A4			Y		
	Σ	6	2	3	1	12

The checksum reflects the number of TRUE combinations covered by each column.

Printer Example



Printer Repair Example (Assume this table is given)

CONDITIONS	VALUES	COMBINATIONS							
		1	2	3	4	5	6	7	8
Printer does not print	Y,N	Y	Y	Y	Y	N	N	N	N
Red-light flashing	Y, N	Y	Y	N	N	Y	Y	N	N
Printer unrecognized	Y, N	Y	N	Y	N	Y	N	Y	N
ACTIONS									
Check power cable?	A1			Y					
printer-computer cable?	A2	Y		Y					
Printer driver installed?	A3	Y		Y		Y		Y	
Check/replace ink	A4	Y	Y			Y	Y		
Check for paper jam	A5		Y		Y				
	Σ								

Can we make any reductions?

Printer Repair Example

CONDITIONS	VALUES	COMBINATIONS							
		1	2	3	4	5	6	7	8
Printer does not print	Y,N	Y	Y	Y	Y	N	N	N	N
Red-light flashing	Y, N	Y	Y	N	N	Y	Y	N	N
Printer unrecognized	Y, N	Y	N	Y	N	Y	N	Y	N
ACTIONS									
Check power cable?	A1			Y					
printer-computer cable?	A2	Y		Y					
Printer driver installed?	A3	Y		Y		Y		Y	
Check/replace ink	A4	Y	Y			Y	Y		
Check for paper jam	A5		Y		Y				
	Σ								

No – because no two combinations have the same result

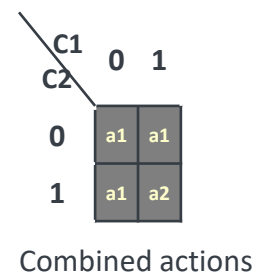
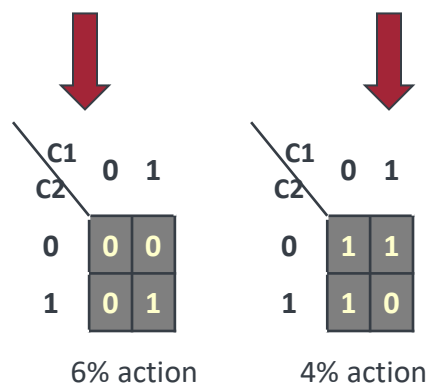
Karnaugh- Maps

Decision Tables map easily to K-Maps



Karnaugh-maps (K-maps): Example 1

- Number each condition
- Replace: Y with 1, and N with 0
- If value for condition = Y, express it as **A** on K-map
- If value for condition = N, express it on K-map as **$\overline{A}$**

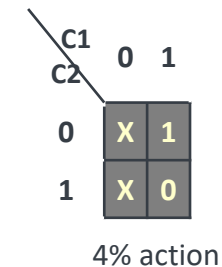


CONDITIONS	ACTIONS
C1. Not Resident C2: Resident, and Gross <= \$15,00	A1: 4% tax A2: 6% tax

	R 1	R 2	R 3	R 4
Conditions				
C1: Resident?	1	1	0	0
C2: Gross > 15 K	1	0	1 $\longleftrightarrow$ 0	
Actions				
Tax %	6%	4%	4%	4%

Representing K-map with "do-not-care" values

	Rule 1	Rule 2	Rule 3
Conditions			
C1: Resident?	Y	Y	N
C2: Gross > 15 K	Y	N	X
Actions			
Tax %	6%	4%	4%



However, it is
always better to use
0/1 not X on k-maps

Different Representations of K-maps

		C3 C4			
		C1 C2	00	01	11
00		a2	a3	a3	
01			a3	a3	
11		a1		a1	
10				a1	

		C1	
		C2	
0	X	0	
1	X	1	

		C3 C4			
		C1 C2	00	01	11
00		1	1	1	
01			1	1	
11		1		1	
10				1	

- We prefer the last representation because it clearly shows the binary predicates.
- For example, you cannot negate a2, you cannot negate X. But you can negate 0/1.
- K-Maps incur a max of one-bit transition.
K-Maps column/row sequence is circular.



Banking System Example with K-map

- For this example, if the relevant CONDITIONS are:
 - C1:** Wrong acct #.
 - C2:** Wrong signature.
 - C3:** There is enough money in the account.
- And the possible ACTIONS are:
 - A1:** Give money.
 - A2:** Print: "insufficient funds".
 - A3:** Call the vigilance dept. & check for fraud.
- But we know this table should have a maximum of $2^3=8$ rules, and we have 5 rules only,

#	1	2	3	4	5	6	7	8	
C1	F	F	F	F	T	T	T	T	
C2	F	F	T	T	F	F	T	T	
C3	F	T	F	T	F	T	F	T	
A1	F	F	?	?	F	?	F	T	
A2	F	F	?	?	T	?	T	F	
A3	T	T	?	?	T	?	F	F	
Σ	1	1	1	1	1	1	1	1	8

C2/C3 C1	00	01	11	10
0				
1			1	

A1

C2/C3 C1	00	01	11	10
0				
1	1			1

A2

C2/C3 C1	00	01	11	10
0	1	1		
1	1			

A3



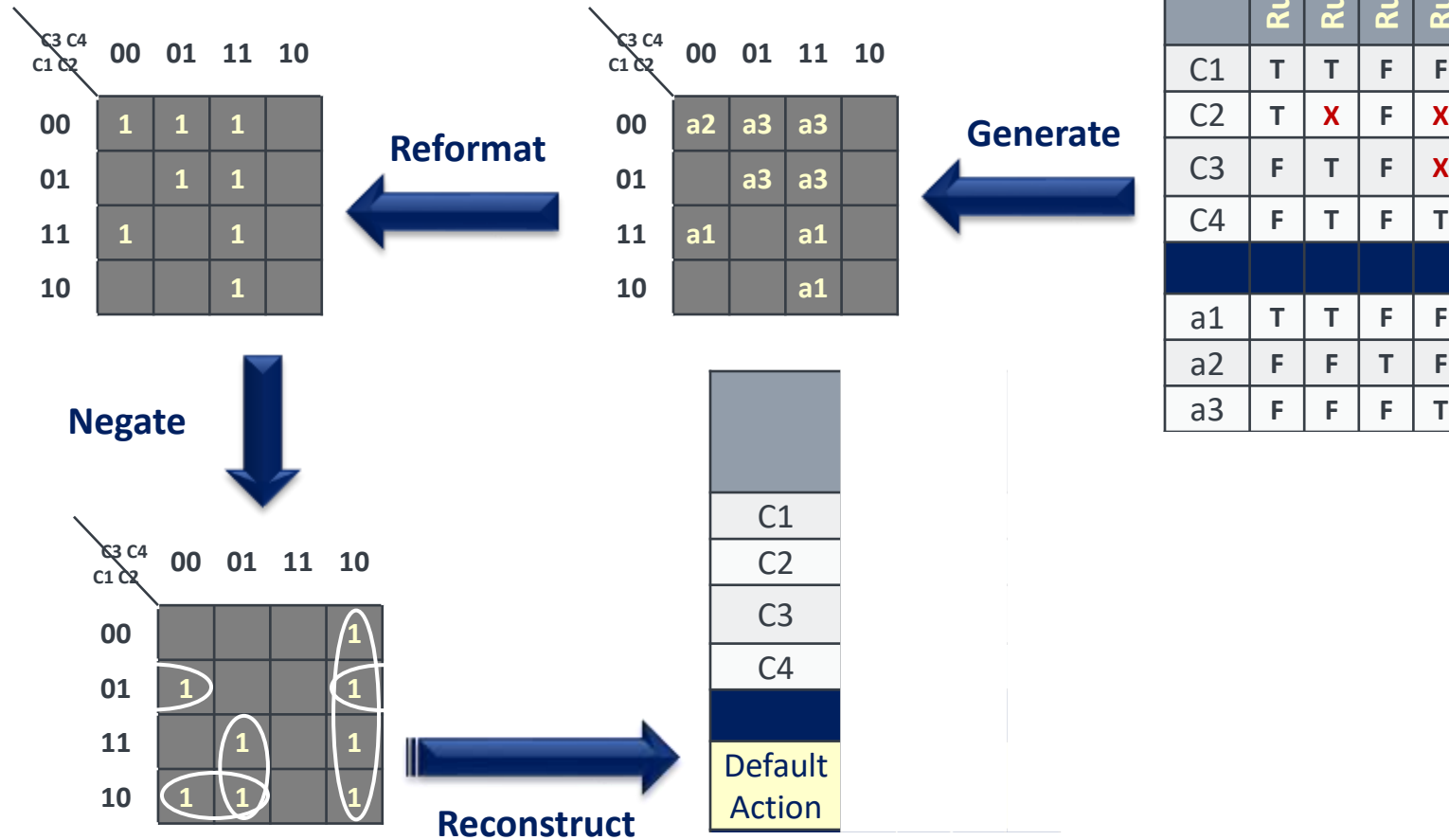
Can you do K-map for action A2?

Default Rules and Actions

- For all rules without specified ACTIONS, *default rules* are presumed to require a *default action* which are generated from a Decision Table.
- We need to develop the set of *default rules* that requires *default action*.
- To calculate the *default rules*:
 - 1) **Fill** out the **Karnaugh map** with explicit entries for the explicitly specified actions.
 - 2) **Negate** the entire chart.
 - 3) Build a **new table** based on the **negation chart**.
 - 4) Present the **equivalent expressions** as a statement of the default rules.
 - Be suspicious of "almost" complete groups of adjacencies.
Eg., If a term contains seven adjacent boxes, question the 8th (missing) box.



Default Rules with Banking System Example



Max # of rules
 $2^4 = 16$
 We have 8 rules
 present; we are
 missing 8

K-map for Banking System Example

Original k-map

		C3 C4 C1 C2	00	01	11	10
00	1	1	1			
01			1	1		
11	1			1		
10				1		

A '1' on k-map corresponds to an existing rule.

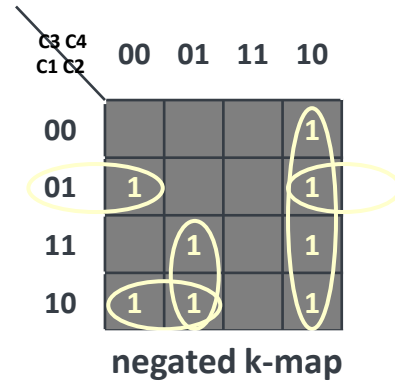
Example: this '1' represents rule 2 with C2 true.

And this '1' represents rule 2 with C2 false.

X expands as '1' or '0'
And this corresponds to two
"next to each other" '1's in
the k-map

	Rule 1	Rule 2	Rule 3	Rule 4
C1	T	T	F	F
C2	T	X	F	X
C3	F	T	F	X
C4	F	T	F	T
a1	T	T	F	F
a2	F	F	T	F
a3	F	F	F	T

K-map Negation Example



So, a total of 8 default rule?

Q: how to find out "do-not-care's"?

A: Count only double ones next to each other.

- Each '1' in the negated k-map represents a rule that is **not** specified in the English spec's. (Which zone is it on S/P/T map?)
- We must consider the unspecified rules in programming and testing.
- Come with an expression for the negated k-map:
- 2 ones next to each other '1' = one variable is **X**.
- 4 ones next to each other '1' = 2 variables are **X**.

Default Rules with Another Example

	Rule 1	Rule 2	Rule 3
C1	T	T	F
C2	T	X	F
C3	F	T	F
C4	F	T	F
a1	F	T	F
a2	F	F	T
a3	T	F	F
#	1	2	1

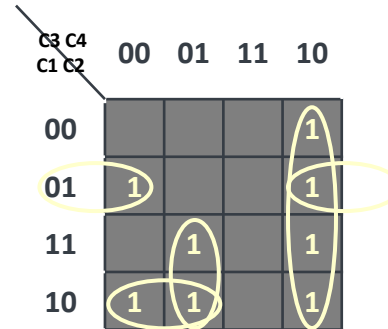
We are missing $2^4 - 4 = 16 - 4 = 12$ rules

Solve to get the default rules, then consolidate

Negated K-map

	$C_1 C_2 C_3 C_4$	
1	FFFF	✓
2	FFFT	✓
3	FFTF	D
4	FFTT	✓
5	FTFF	D
6	FTFT	✓
7	FTTF	D
8	FTTT	✓
9	TFFF	D
10	TFFT	D
11	TFTF	D
12	TFTT	✓
13	TTFF	✓
14	TTFT	D
15	TTTF	D
16	TTTT	✓

D=Default Rule



Source Table

	r_1	r_2	r_3	r_4
C1	T	T	F	F
C2	T	T/F	F	T/F
C3	F	T	F	T/F
C4	F	T	F	T
A1	T	T	F	F
A2	F	F	T	F
A3	F	F	F	T

$$\bar{C}_1 C_2 \bar{C}_4 + C_1 \bar{C}_3 C_4 + C_1 \bar{C}_2 \bar{C}_3 + C_3 \bar{C}_4$$

Default Rules

FTXF
TFFX
TXFT
XXTF

Default Rules

~~C3 C4~~
~~C1 C2~~

	00	01	11	10
00				1
01	1			1
11		1		1
10	1	1		1



	r5	r6	r7	r8
C1	X	F	T	T
C2	X	T	X	F
C3	T	X	F	F
C4	F	F	T	X
Default Action	T	T	T	T

Default Rules

r5 = XXTF

r6 = FTXF

r7 = TXFT

r8 = TFFX

Default Rules with Banking System Example

	r1	r2	r3	r4	
C1	T	T	F	F	
C2	T	X	F	X	
C3	F	T	F	X	
C4	F	T	F	T	
A1	T	T	F	F	
A2	F	F	T	F	
A3	F	F	F	T	
#	1	2	1	4	8



C3 C4 C1 C2	00	01	11	10
00	1	1	1	
01		1	1	
11	1		1	
10			1	



C3 C4 C1 C2	00	01	11	10
00				1
01	1			1
11		1		1
10	1	1		1



	r5	r6	r7	r8	
C1	X	F	T	T	
C2	X	T	X	F	
C3	T	X	F	F	
C4	F	F	T	X	
Default Action	T	T	T	T	
#	4	2	2	2	10



In-Class Examples



nextDate() Limited Entry Decision Table

Table 1

A Limited Entry Decision Table							
#	Condition						
C1	month in M_1 ?						
C2	month in M_2 ?						
C3	month in M_3 ?						
C4	day in D_1 ?						
C5	day in D_2 ?						
C6	day in D_3 ?						
C7	day in D_4 ?						
C8	leap year						
Actions							
A1	impossible						
A2	print next date						

Table 2

A Limited Entry Decision Table							
#	Condition						
C1	month = $\{M_1, M_2, M_3\}$						
C2	day = $\{D_1, D_2, D_3, D_4\}$						
C3	year = leap / non-leap						
Actions							
A1	impossible						
A2	print next date						

Q: Can you calculate the default rules for Table 2 using K-maps?

A: K-maps are only for tables that having all conditions set to binary (T/F). Like Table 1.

What is the max. # of rules in the Decision Table generated by this set of conditions?
If you can consolidate the Decision Table, how many test-cases you'll end-up designing?

Decision Table Based Test-Cases

1. DT testing uses domain Eq. Classes as in domain Eq. Class testing.
2. The sparseness due to the assumption of independence is addressed by careful examination of elements in the cross-product set.
3. For the Eq. Classes defined earlier, the cross product contains 36 elements:

$\langle M_1, D_1, Y_1 \rangle$,	$\langle M_1, D_2, Y_1 \rangle$,	$\langle M_1, D_3, Y_1 \rangle$,	$\langle M_1, D_4, Y_1 \rangle$,
$\langle M_2, D_1, Y_1 \rangle$,	$\langle M_2, D_2, Y_1 \rangle$,	$\langle M_2, D_3, Y_1 \rangle$,	$\langle M_2, D_4, Y_1 \rangle$,
$\langle M_3, D_1, Y_1 \rangle$,	$\langle M_3, D_2, Y_1 \rangle$,	$\langle M_3, D_3, Y_1 \rangle$,	$\langle M_3, D_4, Y_1 \rangle$,
$\langle M_1, D_1, Y_2 \rangle$,	$\langle M_1, D_2, Y_2 \rangle$,	$\langle M_1, D_3, Y_2 \rangle$,	$\langle M_1, D_4, Y_2 \rangle$,
$\langle M_2, D_1, Y_2 \rangle$,	$\langle M_2, D_2, Y_2 \rangle$,	$\langle M_2, D_3, Y_2 \rangle$,	$\langle M_2, D_4, Y_2 \rangle$,
$\langle M_3, D_1, Y_2 \rangle$,	$\langle M_3, D_2, Y_2 \rangle$,	$\langle M_3, D_3, Y_2 \rangle$,	$\langle M_3, D_4, Y_2 \rangle$,
$\langle M_1, D_1, Y_3 \rangle$,	$\langle M_1, D_2, Y_3 \rangle$,	$\langle M_1, D_3, Y_3 \rangle$,	$\langle M_1, D_4, Y_3 \rangle$,
$\langle M_2, D_1, Y_3 \rangle$,	$\langle M_2, D_2, Y_3 \rangle$,	$\langle M_2, D_3, Y_3 \rangle$,	$\langle M_2, D_4, Y_3 \rangle$,
$\langle M_3, D_1, Y_3 \rangle$,	$\langle M_3, D_2, Y_3 \rangle$,	$\langle M_3, D_3, Y_3 \rangle$,	$\langle M_3, D_4, Y_3 \rangle$,

$M1 \in \{\text{Apr, Jun, Sep, Nov}\}$
 $M2 \in \{\text{Jan, Mar, May, Jul, Aug, Oct, Dec}\}$
 $M3 = \text{February}$
 $D1 \in \{1 - 28\}$
 $D2 = 29$
 $D3 = 30$
 $D4 = 31$
 $Y1 = 2000$
 $Y2 = \text{between}(1812-2012)$
 $\quad \& (\text{year} = 0 \bmod 4)$
 $Y3 = \text{between}(1812-2012)$
 $\quad \& (\text{year} \neq 0 \bmod 4)$

4. Notice that many of these are impossible; e.g. $\langle M_1, D_4, -- \rangle$.
5. What happens if you make the day is either $D_1\{1-27\}$, $D_2\{28\}$, $D_3\{29\}$, $D_4\{30\}$, and $D_5\{31\}$?

nextDate() Extended Entry Decision Table

$M1 \in \{\text{Apr, Jun, Sep, Nov}\}$
 $M2 \in \{\text{Jan, Mar, May, Jul, Aug, Oct, Dec}\}$
 $M3 = \text{February}$
 $D1 \in \{1 - 28\}$
 $D2 = 29$
 $D3 = 30$
 $D4 = 31$
 $Y1 = 2000$
 $Y2 = \text{between}(1812-2012) \ \& \ (\text{year} = 0 \bmod 4)$
 $Y3 = \text{between}(1812-2012) \ \& \ (\text{year} \neq 0 \bmod 4)$

An Extended Entry Decision Table							
#	Condition						
C1	month in $M_1, M_2, M_3?$						
C2	day in $D_1, D_2, D_3, D_4?$						
C3	year in $Y_1, Y_2, Y_3?$						
	Actions						
A1	impossible						
A2	increment date						
A3	reset day (to 1)						
A4	increment month						
A5	reset month (to Jan)						
A6	increment year						

Q: Can you calculate the default rules for this table using K-maps?

A: No, the provided mutually exclusive conditions cannot be negated.

Now assume those conditions;

What is the max. # of rules in the DT generated by this set of conditions / actions?
 Use information from previous slide to fill the table.

Another **nextDate()** Decision Table

- Assume the example utilizes the following set of conditions shown here;
 - Calculate the max # of rules in the DT, and;
 - Develop the Decision Table for the following actions:

Actions

A1: impossible()
A2: incDay()
A3: resetDay()
A4: incMonth()
A5: resetMonth()
A6: incYear()

Conditions

M1 $\in$ {Apr, Jun, Sep, Nov}
M2 $\in$ {Jan, Mar, May, Jul,
Aug, Oct}
M3 = February
M4 = December
D1 $\in$ {1 - 27}
D2 = 28
D3 = 29
D4 = 30
D5 = 31
Y1/Y2 = isLeap()



nextDate() Extended Entry Decision Table

Actions

A1: impossible()
A2: incDay()
A3: resetDay()
A4: incMonth()
A5: resetMonth()
A6: incYear()

Conditions

M1 ∈ {Apr, Jun, Sep, Nov}
M2 ∈ {Jan, Mar, May, Jul, Aug, Oct}
M3 = February
M4 = December
D1 ∈ {1 - 27}
D2 = 28
D3 = 29
D4 = 30
D5 = 31
Y1/Y2 = isLeap()

Final Version																							
#	Condition	01-02	03-04	05-06	07-08	09-10	11-12	13-14	15-16	17-18	19-20	21-22	23-24	25-26	27-28	29-30	31-32	33	34	35	36	37-38	39-40
C1	month in? {M ₁ -M ₄ }	Can you fill the Decision Table?																					
C2	day in? {D1-D5}																						
C3	year in (leap)?																						
	Actions																						
A1	impossible																						
A2	incDay()																						
A3	resetDay(to 1)																						
A4	incMonth()																						
A5	resetMonth()																						
A6	incYear()																						

Notice there are 40 rules in this Decision Table, corresponding to the 40 elements of the cross product of the revised equivalence classes



Decision Table FunAir Exercise

Forever-United National Airlines (FunAir) offers the cities of Miami and New York as its destinations from Germany. FunAir offers a percentage rebate (from the standard airfare). The percentage rebate is determined from a Decision Table based on the destination, the duration of the stay, and the age of the passenger. The following conditions apply:

- 1) Infants under 2 years old fly free accompanied by adults
- 2) Passengers who are between 2 and 18 years old get a 50% rebate
- 3) Adult passengers get a 20% rebate when flying to Miami if the departure is not between December 21st and December 30th (inclusive), and as long as the duration of stay does not exceed 6 days. There is considered the Miami holiday rate.
- 4) There is no holiday rate for New York
- 5) Holiday rate applies for adults only.
- 6) When no rebate is offered, it should be calculated as a zero rebate.

FunAir Exercise Requirements

You are required to:

- 1) Define all variables.
- 2) Drive the equivalence classes for all input variables and summarize the given conditions using defined variables.
- 3) Develop the Decision Table
- 4) Reduce the Decision Table if possible, and then calculate the number of test-cases.
- 5) Using K-maps, define the default rules (if any)
- 6) Comment on the completeness and consistency of the given specifications.

Comment on the following table;

Columns 2 and 8 are redundant, we can eliminate.

This table effectively has $19-1-2=16$ columns.

COND.	VALUES	COMBINATIONS							
		1	2	3	4	5	6	7	8
C1: {Y, N}		Y	Y	Y	X	N	X	N	X
C2: {S, M, D, W}		S	M	D	W	S	M	D	W
C3: {C, A, I}		C	A	X	X	C	A	X	I
ACTIONS									
A1				✓				✓	
A2			✓				✓		
A3		✓			✓	✓		✓	✓
A4					✓				✓
A5			✓				✓		
Σ		1	1	3	6	1	2	3	2
		19/24							

The table has redundancies:

Col. # 4: {Y/N, W, C/A/I} similar to Col. # 8: {Y/N, W, I}

Col. # 2: {Y, M, A}

similar to

Col. # 6: {Y/N, M, A}

Col. # 4: {Y/N, W, C/A/I} similar to

Col. # 8: {Y/N, W, I}

The previous DT after removing redundancies

Columns 1 & 5 can be reduced to a single column.

COND.	VALUES	COMBINATIONS					
		1	2	3	4	5	6
C1: {Y, N}		Y	X	Y	X	N	N
C2: {S, M, D, W}		S	M	D	W	S	D
C3: {C, A, I}		C	A	X	X	C	X
ACTIONS							
A1				✓			✓
A2			✓				
A3		✓			✓	✓	✓
A4					✓		
A5			✓				
Σ		1	2	3	6	1	3
		16/24					

The previous DT after combining columns #1 and #5

		VALUES	COMBINATIONS				
COND.			1	2	3	4	5
C1: {Y, N}			Y N	X	Y	X	N
C2: {S, M, D, W}			S	M	D	W	D
C3: {C, A, I}			C	A	X	X	X
ACTIONS							
A1					✓		✓
A2				✓			
A3			✓			✓	✓
A4						✓	
A5				✓			
Σ			2	2	3	6	3

No
un
ass

16/24

No more reduction is possible. Only unique actions are associated with each column.

You cannot use K-Map to develop the Default Rule here (non-binary conditions).

Discussion





THANK YOU

Stevens Institute of Technology
1 Castle Point Terrace, Hoboken, NJ 07030